Deploying to AWS Instead of DigitalOcean

In this chapter, we've created a three-node Swarm cluster running on DigitalOcean. However, you may be thinking, "That's all very well, but what about deploying to (insert your cloud provider of choice)?" Good question.

Before we close out the chapter, we're going to see what it would take to deploy our app to a second cloud provider: AWS. The process is very similar to that for DigitalOcean; there are just two or three key differences.

Here are the steps:

1. Sign up for an AWS account if you don't already have one, and set the AWS environment variables:

   ```
   export AWS_ACCESS_KEY_ID=<your access key id>
   export AWS_SECRET_ACCESS_KEY=<your secret access key>
   export AWS_DEFAULT_REGION=<your default region>
   ```

2. Create the manager instance:

   ```
   $ docker-machine create \
   >     --driver amazonec2 \
   >     --amazonec2-open-port 80 \
   >     --amazonec2-open-port 8080 \
   >     --amazonec2-region eu-west-2 \
   >     aws-manager-1
   ```

   Note that we use the AWS-specific --amazonec2-open-port option to open up the firewall to allow us to hit ports 80 (for web) and 8080 (for visualizer).

3. Get the manager's internal IP address as before, using:

   ```
   $ docker-machine ssh aws-manager-1 "ifconfig eth0"
   ```

4. Add the user to the docker group:

   ```
   $ docker-machine ssh aws-manager-1 \
       'sudo usermod -a -G docker $USER'
   ```

   With DigitalOcean, docker-machine's SSH session is configured to connect as the root user. However, with the AWS driver, we connect via SSH as the ubuntu, non-root user. In order to have permissions to run Docker commands, we add the ubuntu user to the docker group.[13]

5. Initialize the swarm:

   ```
   $ docker-machine ssh aws-manager-1 \
       "docker swarm init --advertise-addr 172.31.29.132"
   ```

13. https://docs.docker.com/install/linux/linux-postinstall/#manage-docker-as-a-non-root-user